

4-4장

2101080 정진용

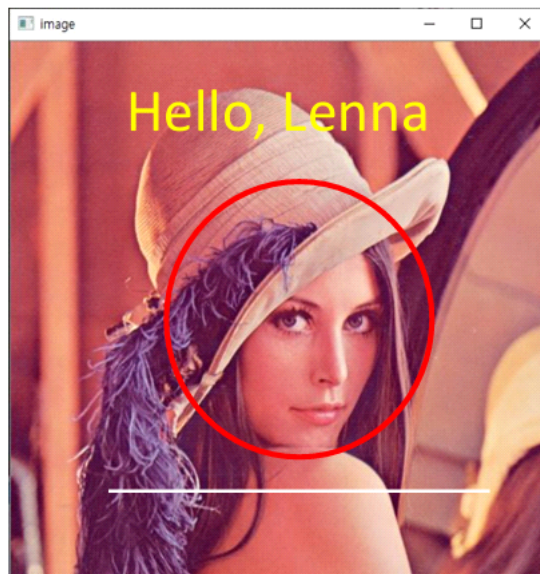
1.

(ANSWER)

실습과제1



- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 l(엘)을 누르면 얼굴 아래에 직선을 그리고 c를 누르면 얼굴주위에 원을 그리고 t를 누르면 문자열을 영상 상단에 출력하고 기본 구조는 무한 반복문으로 작성하고 q를 누르면 종료하는 코드를 작성 하시오.



코드:

```
#include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 입출력을 위한 표준 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용

int main() {
    Mat image = imread("lenna.bmp"); // lenna.bmp 이미지를 Mat 형식으로 불러옴
    if (image.empty()) {cerr << "image load failed " << endl; return -1;} // 이미지가 비어있으면 에러 메시지 출력 후 종료

    int key; // 키 입력을 저장할 변수 선언
    while (true) {
        imshow("image", image); // 이미지 출력
        key = waitKey(); // 키 입력 대기

        Size textsz = getTextSize("Hello, Lenna", FONT_HERSHEY_COMPLEX, 2,1,0); // 텍스트의 크기 계산

        if (key == 'l') {
            line(image, Point(132, 415), Point(418, 415), Scalar(255, 255, 255), 2); // 흰색 직선을 그림
        }
        else if (key == 'c') {
            circle(image, Point(282, 263), 130, Scalar(0, 0, 255),3); // 빨간 원을 그림
        }
        else if (key == 't') {
            putText(image, "Hello, Lenna", Point(((image.cols - textsz.width) / 2), textsz.height * 2), FONT_HERSHEY_COMPLEX, 2, Scalar(0, 255, 255), 2, 1); // 이미지 중앙에 노란 텍스트 출력
        }
        else if (key == 'q') break; // q 키를 누르면 프로그램 종료
    }
}
```

콘솔: 동영상 파일로 제출하겠습니다

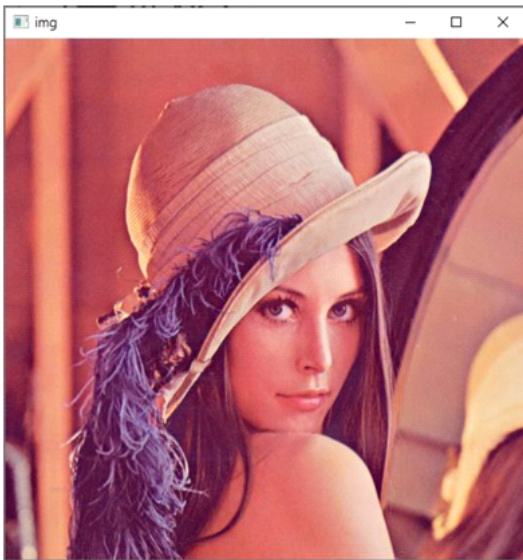
2.

(ANSWER)

실습과제2



- Lenna 영상 위에서 발생하는 마우스 이벤트의 횟수를 세어서 출력하는 프로그램을 작성하시오. 왼쪽버튼 down, 왼쪽버튼 up, 마우스 move 이벤트만 발생횟수를 발생할 때마다 화면에 다음처럼 출력하라.



```
D:\Users\2sungryul\Dropbox\Lecture\2021년2학...  
EVENT_MOUSEMOVE: 223  
EVENT_LBUTTONDOWN: 12  
EVENT_LBUTTONDOWN: 12  
EVENT_LBUTTONDOWN: 13  
EVENT_MOUSEMOVE: 224  
EVENT_LBUTTONDOWN: 13  
EVENT_LBUTTONDOWN: 14  
EVENT_MOUSEMOVE: 225  
EVENT_MOUSEMOVE: 226  
EVENT_MOUSEMOVE: 227  
EVENT_MOUSEMOVE: 228  
EVENT_MOUSEMOVE: 229  
EVENT_MOUSEMOVE: 230  
EVENT_MOUSEMOVE: 231  
EVENT_MOUSEMOVE: 232
```

코드:

```
#include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 입출력을 위한 표준 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용

void onMouse(int event, int x, int y, int flags, void* userdata); // 마우스 이벤트 콜백 함수 선언
int main() {
    Mat image = imread("lenna.bmp"); // lenna.bmp 이미지를 Mat 형식으로 불러옴
    if (image.empty()) {
        cerr << "image load failed" << endl; // 이미지 불러오기 실패 시 에러 메시지 출력
        return -1; // 프로그램 종료
    }

    int key; // 키 입력 저장 변수 선언
    namedWindow("img"); // "img"라는 이름의 윈도우 생성
    setMouseCallback("img", onMouse); // 마우스 콜백 함수 등록

    while (true) {
        imshow("img", image); // 이미지 출력
        key = waitKey(50); // 50ms 동안 키 입력 대기
        if (key == 'q') break; // 'q' 키 입력 시 루프 종료
    }

    void onMouse(int event, int x, int y, int flags, void* userdata) { // 마우스 이벤트 콜백 함수 정의
        static int lu = 0, ld = 0, mm = 0; // 마우스 이벤트 발생 횟수를 저장하는 정적 변수 (버튼 업, 다운, 이동)
        switch (event) {
            case EVENT_LBUTTONDOWN: // 마우스 왼쪽 버튼 누를 때
                cout << "EVENT_LBUTTONDOWN: " << ++ld << endl; // 누른 횟수 출력
                break;
            case EVENT_LBUTTONUP: // 마우스 왼쪽 버튼 땔 때
                cout << "EVENT_LBUTTONUP: " << ++lu << endl; // 땔 횟수 출력
                break;
            case EVENT_MOUSEMOVE: // 마우스 이동할 때
                cout << "EVENT_MOUSEMOVE: " << ++mm << endl; // 이동 횟수 출력
                break;
        }
    }
```

콘솔: 동영상 파일로 제출하겠습니다.

3.

(ANSWER)

실습과제3

컴퓨터 비전과
머신러닝



- Lenna 영상을 그레이스케일 영상으로 변환 후 화면에 출력하고 마우스로 클릭하는 화면 좌표와 그점에서 화소값을 출력하는 프로그램을 작성하시오. 주의: 마우스 좌표계와 Mat::at 함수의 매개변수를 잘 연결할 것

```
D:\Users\2sungryul\Dropbox\Lecture\2019년2학기\프로그...
좌표: (367,65), 화소값: 23
좌표: (385,272), 화소값: 233
좌표: (21,22), 화소값: 174
좌표: (19,380), 화소값: 176
좌표: (273,376), 화소값: 218
좌표: (216,207), 화소값: 210
좌표: (254,200), 화소값: 167
좌표: (195,97), 화소값: 200
```



코드:

```
#include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 입출력을 위한 표준 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용

void onmouse(int event, int x, int y, int flags, void* userdata); // 마우스 이벤트 콜백 함수 선언

int main() {
    Mat image = imread("lenna.bmp", IMREAD_GRAYSCALE); // "lenna.bmp" 이미지를 명암도(그레이스케일)로 읽어와 Mat 객체에 저장
    if (image.empty()) {
        cerr << "image load failed" << endl; // 이미지 로딩 실패 시 오류 메시지 출력
        return -1; // 프로그램 종료
    }

    int key; // 키 입력 저장 변수 선언
    namedWindow("img"); // "img"라는 이름의 윈도우 생성
    setMouseCallback("img", onmouse, &image); // 마우스 콜백 함수 등록, 이미지 포인터를 userdata로 전달

    while (true) {
        imshow("img", image); // 현재 이미지 출력
        key = waitKey(10); // 10ms 동안 키 입력 대기
        if (key == 'q') break; // 'q' 키 입력 시 루프 종료
    }

    void onmouse(int event, int x, int y, int flags, void* userdata) { // 마우스 이벤트 콜백 함수 정의

        Mat* image = (Mat*)userdata; // 전달된 void 포인터를 Mat 포인터로 형변환
        switch (event) {
            case EVENT_LBUTTONDOWN: // 마우스 왼쪽 버튼을 누를 경우
                cout << "좌표: " << Point(x, y) << ", 화소값:" << (int)(*image).at<uchar>(y, x) << endl; // 클릭한 좌표와 해당 위치의 화소값 출력
                break;
        }
    }
}
```

콘솔: 동영상 파일로 제출하겠습니다.

4.

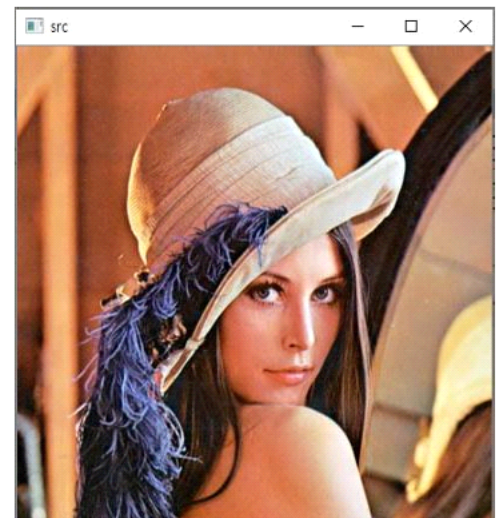
(ANSWER)

실습과제4



- Lenna 영상을 컬러 영상으로 화면에 출력하고 마우스로 클릭하는 화면 좌표와 해당 좌표의 화소값(B,G,R)을 출력하는 프로그램을 작성하시오. 주의: 마우스 좌표계와 Mat::at함수의 매개변수를 잘 연결할 것

```
D:\Users\2sungryul\Dropbox\Lecture\2019년2학기\프로그...
좌표 : (395,19), 화소값(B,G,R) : (16,20,31)
좌표 : (394,262), 화소값(B,G,R) : (198,235,239)
좌표 : (12,15), 화소값(B,G,R) : (100,151,224)
좌표 : (21,395), 화소값(B,G,R) : (194,215,213)
좌표 : (254,345), 화소값(B,G,R) : (166,199,232)
좌표 : (216,203), 화소값(B,G,R) : (5,14,34)
좌표 : (185,94), 화소값(B,G,R) : (174,195,222)
좌표 : (175,169), 화소값(B,G,R) : (164,121,130)
좌표 : (315,166), 화소값(B,G,R) : (12,22,39)
```



코드:

```
#include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 입출력을 위한 표준 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용

void onMouse(int event, int x, int y, int flags, void* userdata); // 마우스 이벤트 콜백 함수 선언

int main() {
    string title = "src"; // 윈도우 제목 설정
    int key; // 키보드 입력을 저장할 변수
    Mat image = imread("lenna.bmp"); // lenna.bmp 컬러 이미지를 불러와 Mat 객체에 저장
    if (image.empty()) {
        cerr << "image load failed" << endl; // 이미지 로딩 실패 시 에러 메시지 출력
        return -1; // 프로그램 종료
    }

    namedWindow(title); // 지정한 제목으로 윈도우 창 생성
    setMouseCallback(title, onMouse, &image); // 마우스 이벤트 콜백 함수 등록, 이미지 주소를 userdata로 전달

    while (true) {
        imshow(title, image); // 이미지 출력
        key = waitKey(50); // 50ms 동안 키 입력 대기
        if (key == 'q') break; // 'q' 키 입력 시 루프 종료
    }

    void onMouse(int event, int x, int y, int flags, void* userdata) { // 마우스 이벤트 콜백 함수 정의
        switch (event) {
            case EVENT_LBUTTONDOWN: // 마우스 왼쪽 버튼을 누른 경우
                cout << "좌표: " << Point(x, y) << ", 화소값(B,G,R): " << *(Mat*)userdata.at<Vec3b>(y, x) << endl; // 클릭한 위치의 좌표와 BGR 화소값 출력
            }
        }
```

콘솔: 동영상 파일로 제출하겠습니다.

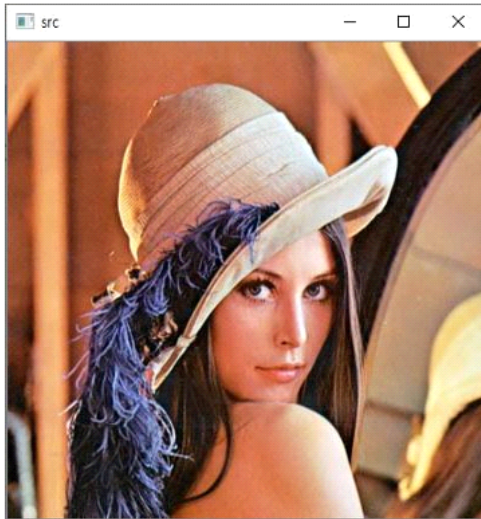
5.

(ANSWER)

실습과제5



- Lenna 영상을 컬러 영상으로 화면에 출력하고 마우스로 드래깅 한 영역을 추출하여 화면에 출력하는 프로그램을 작성 하시오. 코드 4-11을 참고할 것



레나 화면에서 마우스로
드래깅한 영역임

코드:

```
#include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 입출력을 위한 표준 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용

void onMouse(int event, int x, int y, int flags, void* userdata); // 마우스 이벤트 콜백 함수 선언

int main() {
    string title = "src"; // 메인 윈도우 이름 설정
    Mat image = imread("lenna.bmp"); // lenna.bmp 이미지를 컬러로 읽어서 Mat 객체에 저장
    if (image.empty()) {
        cerr << "image load failed" << endl; // 이미지 로딩 실패 시 에러 메시지 출력
        return -1; // 프로그램 종료
    }

    namedWindow(title); // "src"라는 이름의 윈도우 생성
    setMouseCallback(title, onMouse, &image); // 마우스 이벤트 콜백 함수 등록, 이미지 주소 전달

    imshow(title, image); // 원본 이미지 출력
    waitKey(); // 키 입력이 있을 때까지 대기

    return 0; // 프로그램 종료
}

void onMouse(int event, int x, int y, int flags, void* userdata) { // 마우스 이벤트 콜백 함수 정의

    static Point xx; // 드래그 시작 좌표를 저장할 변수 (정적 변수로 선언하여 상태 유지)
    switch (event) {
        case EVENT_LBUTTONDOWN: // 마우스 왼쪽 버튼을 누른 경우
            xx = Point(x, y); // 시작 좌표 저장
            break;
        case EVENT_LBUTTONUP: // 마우스 왼쪽 버튼을 땀 경우
            // 드래그한 영역을 잘라서 새로운 창에 출력
            imshow("src1", (*(Mat*)userdata)(Rect(Point(min(xx.x,x),min(xx.y,y)), Point(max(xx.x, x), max(xx.y, y))))); // 시작점과 끝점을 기준으로 사각형 영역 지정 후 출력
            break;
    }
}
```

콘솔: 동영상으로 제출하겠습니다.

6.

(ANSWER)

실습과제6



- 크기가 400X200, 배경색이 백색인 컬러영상을 생성하고 마우스 왼쪽 버튼을 누르면 배경색이 Red가 되고 마우스 오른쪽 버튼을 누르면 배경색이 Blue가 되도록 프로그램을 작성하시오.



초기상태



마우스 왼쪽버튼 클릭



마우스 오른쪽버튼 클릭

코드:

```
//include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 표준 입출력 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용
void onMouse(int event, int x, int y, int flags, void* userdata); // 마우스 이벤트 콜백 함수 선언

int main() {
    Mat image(Size(400, 200), CV_8UC3, Scalar(255, 255, 255)); // 400x200 크기의 흰색 배경 이미지 생성 (3채널 컬러)
    int key; // 키 입력 저장 변수
    namedWindow("src"); // "src"라는 이름의 윈도우 생성
    setMouseCallback("src", onMouse, &image); // 마우스 이벤트 콜백 함수 등록, 이미지 주소 전달

    while (true) {
        imshow("src", image); // 이미지 출력
        key = waitKey(50); // 50ms 동안 키 입력 대기
        if (key == 'q') break; // 'q' 키 입력 시 반복 종료
    }

    return 0; // 프로그램 정상 종료
}

void onMouse(int event, int x, int y, int flags, void* userdata) {
    switch (event) {
        case EVENT_LBUTTONDOWN: // 왼쪽 마우스 버튼 클릭 시
            *(Mat*)userdata = Scalar(0, 0, 255); // 이미지 전체를 빨간색으로 채움 (BGR 순서)
            break;
        case EVENT_RBUTTONDOWN: // 오른쪽 마우스 버튼 클릭 시
            *(Mat*)userdata = Scalar(255, 0, 0); // 이미지 전체를 파란색으로 채움 (BGR 순서)
            break;
    }
}
```

콘솔:동영상으로 결과 제출하겠습니다

7.

(ANSWER)

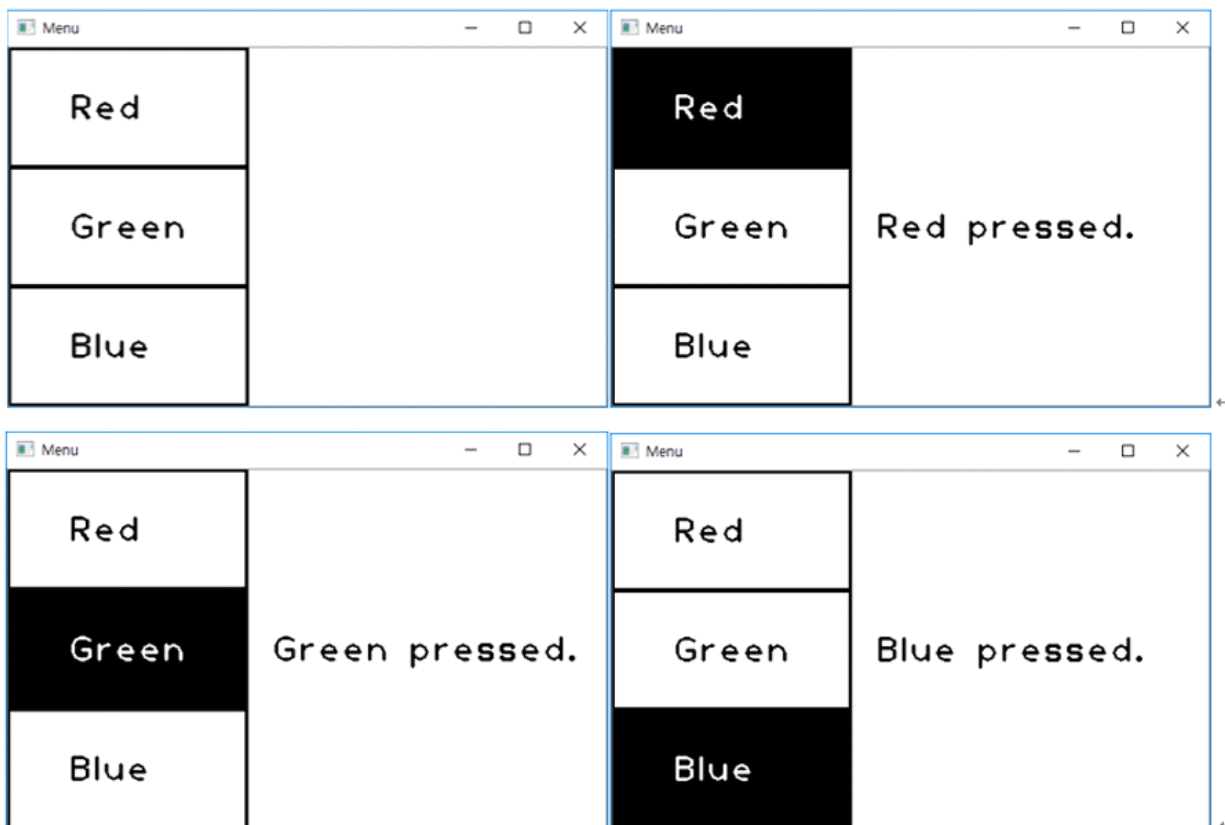
실습과제 7



● 다음과 같이 동작하는 프로그램을 작성하시오.

- ① 초기화면은 좌측상단 그림처럼 작성할 것. 전체화면은 폭 500픽셀, 높이 300픽셀, 메뉴박스의 크기는 폭 200픽셀, 높이 100픽셀, 문자의 폰트와 위치는 적당히 보기 좋게 조정할 것
- ② Red 사각형 안에서 마우스 좌측버튼을 클릭하면 우측상단 그림처럼 실행되어야 함
- ③ Green 사각형 안에서 마우스 좌측버튼을 클릭하면 좌측하단 그림처럼 실행되어야 함
- ④ Blue 사각형 안에서 마우스 좌측버튼을 클릭하면 우측하단 그림처럼 실행되어야 함

실습과제 7



코드:

```
#include "opencv2/opencv.hpp" // OpenCV 기본 헤더 파일 포함
#include <iostream> // 입출력을 위한 표준 라이브러리 포함
using namespace std; // std:: 생략을 위해 사용
using namespace cv; // cv:: 생략을 위해 사용

void onMouse(int event, int x, int y, int flags, void* userdata); // 마우스 이벤트 콜백 함수 선언

int main() {
    // 300x500 크기의 흰색 배경 이미지를 생성 (3채널 컬러 이미지)
    Mat image(300, 500, CV_8UC3, Scalar(255, 255, 255));
    String text[3] = { "Red", "Green", "Blue" }; // 버튼 텍스트 배열
    Rect rec[3]; // 버튼 영역을 저장할 배열
    Size textSize; // 텍스트 크기
    int clicked = -1; // 클릭된 버튼 상태를 기억하는 변수, 기본값은 -1 (클릭되지 않음)

    // 버튼 영역 및 텍스트 위치 설정
    for (int i = 0; i < 3; i++) {
        rec[i] = Rect(0, i * 100, 200, 100); // 각 버튼의 영역 설정 (100px 간격)
        rectangle(image, rec[i], Scalar(0, 0, 0), 2); // 버튼 영역에 검은색 테두리 그리기
        textSize = getTextSize(text[i], FONT_HERSHEY_COMPLEX, 1, 1, 0); // 텍스트 크기 계산
        Point textOrg((rec[i].width - textSize.width) / 2, // 텍스트를 버튼 영역의 가운데로 배치
            rec[i].y + (rec[i].height + textSize.height) / 2);
        putText(image, text[i], textOrg, FONT_HERSHEY_COMPLEX, 1, Scalar(0, 0, 0), 1); // 버튼에 텍스트 삽입
    }

    namedWindow("Menu"); // "Menu"라는 이름의 윈도우 생성
    setMouseCallback("Menu", onMouse, &clicked); // 마우스 이벤트 콜백 함수 설정

    while (true) {
        // 클릭된 버튼 상태에 따라 이미지 업데이트
        Mat tempImage = image.clone(); // 원본 이미지를 복사하여 업데이트 처리
        if (clicked != -1) { // 클릭된 상태가 있을 때만 갱신
            rectangle(tempImage, rec[clicked], Scalar(0, 0, 0), -1); // 클릭된 버튼만 검정색 배경
            putText(tempImage, text[clicked], Point((rec[clicked].width - getTextSize(text[clicked], FONT_HERSHEY_COMPLEX, 1, 1, 0).width) /
                2, rec[clicked].y + (rec[clicked].height + getTextSize(text[clicked], FONT_HERSHEY_COMPLEX, 1, 1, 0).height) / 2), FONT_HERSHEY_COMPLEX, 1, Scalar(255, 255, 255), 2);
            String msg = text[clicked] + " pressed"; // 클릭된 버튼 메시지 생성
            putText(tempImage, msg, Point(200 + (300 - getTextSize(msg, FONT_HERSHEY_COMPLEX, 1, 1, 0).width) / 2, (300 + getTextSize(msg, FONT_HERSHEY_COMPLEX, 1, 1, 0).height) / 2),
                FONT_HERSHEY_COMPLEX, 1, Scalar(0, 0, 0), 2); // 메시지 출력

            imshow("Menu", tempImage); // 업데이트된 이미지를 "Menu" 창에 출력
            if (waitKey(50) == 'q') break; // 'q' 키로 종료
        }

        return 0; // 프로그램 종료
    }

    void onMouse(int event, int x, int y, int flags, void* userdata) { // 마우스 이벤트 콜백 함수 정의
        if (event != EVENT_LBUTTONDOWN) return; // 왼쪽 버튼 클릭이 아니면 무시

        int* clicked = (int*)userdata; // 클릭된 버튼 상태를 저장할 변수 포인터
        Rect rec[3] = { // 버튼 영역 정의
            Rect(0, 0, 200, 100),
            Rect(0, 100, 200, 100),
            Rect(0, 200, 200, 100)
        };

        for (int i = 0; i < 3; i++) { // 각 버튼 영역에 대해 클릭 여부 확인
            if (rec[i].contains(Point(x, y))) { // 마우스 클릭 위치가 버튼 영역 내에 있으면
                *clicked = i; // 클릭된 버튼 인덱스 저장
            }
        }
    }
}
```

콘솔:동영상 파일로 제출하겠습니다